

Web 应用安全防护最佳实践指南

Web 应用安全防护最佳实践指南

文档类型：安全最佳实践 版本：v2.0 适用范围：Web 应用 / API 服务 标准参考：OWASP Top 10:2025 / CWE Top 25**文档类型**：安全最佳实践

版本：v2.0

适用范围：Web 应用 / API 服务

标准参考：OWASP Top 10:2025 / CWE Top 25

文档类型：安全最佳实践 版本：v2.0 适用范围：Web 应用 / API 服务 标准参考：OWASP Top 10:2025 / CWE Top 25**文档类型**：安全最佳实践

版本：v2.0

适用范围：Web 应用 / API 服务

标准参考：OWASP Top 10:2025 / CWE Top 25

1. 概述

本指南覆盖 Web 应用安全防护的 8 个核心领域，每个领域包含攻击原理、防护方案和代码示例。遵循本指南可有效防御 OWASP Top 10 中的大多数威胁。

1.1 安全防护层次

—
—
—
—
—

— WAF / CDN
—
—

—
—
—

2. 输入验证与输出编码

2.1 XSS（跨站脚本）防护

攻击原理：攻击者将恶意脚本注入页面，在受害者浏览器中执行。

防护方案：

```
// 1. HTML
function escapeHtml(unsafe: string): string {
  return unsafe
    .replace(/&/g, "&amp;")
    .replace(/</g, "&lt;")
    .replace(/>/g, "&gt;")
    .replace(/"/g, "&quot;")
    .replace(/'/g, "&#039;");
}

// 2. CSP
//
// Content-Security-Policy: default-src 'self'; script-src 'self' 'nonce-{random}'
app.use((req, res, next) => {
  const nonce = crypto.randomBytes(16).toString("base64");
  res.locals.nonce = nonce;
  res.setHeader(
    "Content-Security-Policy",
    `default-src 'self'; script-src 'self' 'nonce-${nonce}'`
  );
  next();
});

// 3. DOM API
//
element.innerHTML = userInput;

//
element.textContent = userInput;
```

2.2 SQL 注入防护2.2 SQL 注入防护

方法方法	安全等级安全等级	说明说明
参数化查询参数化查询	最高 最高	使用预编译语句 + 参数绑定使用预编译语句 + 参数绑定
ORM 框架ORM 框架	高 高	需确保 ORM 正确使用参数化需确保 ORM 正确使用参数
输入过滤输入过滤	中 中	仅作辅助手段，不应作为唯一防护仅作辅助手段，不应作为
字符串拼接字符串拼接	危险 危险	绝对禁止绝对禁止

```
// PreparedStatementJava
String customerName = request.getParameter("customerName");
String query = "SELECT account_balance FROM user_data WHERE user_name = ?";
PreparedStatement pstmt = connection.prepareStatement(query);
pstmt.setString(1, customerName); //
ResultSet results = pstmt.executeQuery();
```

3. 认证与会话管理3. 认证与会话管理

3.1 密码安全策略3.1 密码安全策略

策略策略	要求要求	实现方式实现方式
密码强度密码强度	最少 8 位, 含大小写字母+数字+特殊字符	前端 + 后端双重校验前端 + 后端双重校验
哈希算法哈希算法	bcrypt / argon2idbcrypt / argon2id	cost factor ≥ 12cost factor ≥ 12
密码历史密码历史	禁止重复使用最近 5 次禁止重复使用最近 5 次	维护密码历史表维护密码历史表
暴力破解防护暴力破解防护	5 次失败锁定 15 分钟5 次失败锁定 15 分钟	Redis 计数器 + TTLRedis 计数器 + TTL

```
import bcrypt

def hash_password(password: str) -> str:
    """ bcrypt """
    salt = bcrypt.gensalt(rounds=12)
    return bcrypt.hashpw(password.encode('utf-8'), salt).decode('utf-8')

def verify_password(password: str, hashed: str) -> bool:
    """ """
    return bcrypt.checkpw(password.encode('utf-8'), hashed.encode('utf-8'))
```

3.2 会话安全3.2 会话安全

```
// Cookie
const sessionConfig = {
  cookie: {
    httpOnly: true,      // JavaScript
    secure: true,        // HTTPS
    sameSite: 'strict',  //
    maxAge: 3600000,     // 1
    domain: '.example.com',
    path: '/',
  },
  name: '__Host-Session', // __Host- Secure
  resave: false,
  saveUninitialized: false,
  rolling: true,          //
};
```

4. 访问控制4. 访问控制

4.1 权限模型设计4.1 权限模型设计

RBAC

```
super_admin ()
└─ admin ()
    └─ editor ()
    └─ viewer ()
```

```
resources: { user, article, report, system }
actions: { create, read, update, delete }
```

	user:*	article:*	report:read	system:read
super_admin	CRUD	CRUD		
admin	CRUD	CRUD		
editor	R	CRU		
viewer	R	R		

4.2 接口级鉴权4.2 接口级鉴权

```
// Go
func RequirePermission(resource, action string) gin.HandlerFunc {
    return func(c *gin.Context) {
        user := c.MustGet("user").(*User)

        if !user.HasPermission(resource, action) {
            c.JSON(403, gin.H{
                "error": "Forbidden",
                "detail": fmt.Sprintf(
                    "User lacks permission: %s:%s",
                    resource, action,
                ),
            })
            c.Abort()
            return
        }
        c.Next()
    }
}

//
router.PUT("/api/articles/:id",
    RequirePermission("article", "update"),
    updateArticle,
)
```

5. 加密与密钥管理5. 加密与密钥管理

5.1 传输层安全5.1 传输层安全

```
# Nginx TLS 1.3
server {
    listen 443 ssl http2;
    server_name example.com;

    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384;
    ssl_prefer_server_ciphers off;
    ssl_ecdh_curve X25519:secp384r1;

    ssl_certificate /etc/nginx/certs/fullchain.pem;
    ssl_certificate_key /etc/nginx/certs/privkey.pem;
    ssl_stapling on;
    ssl_stapling_verify on;

    add_header Strict-Transport-Security "max-age=63072000; includeSubDomains; preload" always;
    add_header X-Content-Type-Options "nosniff";
    add_header X-Frame-Options "DENY";
}
```

5.2 应用层加密5.2 应用层加密

场景场景	推荐算法推荐算法	密钥长度密钥长度	说明说明
对称加密对称加密	AES-256-GCM	AES-256-GCM 256 bit256 bit	自带认证自带认证
非对称加密非对称加密	RSA-4096 / ECDSA P-384	RSA ——	密钥交换/签名密钥交换/签名
哈希哈希	SHA-256 / SHA-384	SHA-256 ——	避免 MD5/SHA-1避免 MD5/SHA-1
密码哈希密码哈希	Argon2id	Argon2id ——	内存硬哈希函数内存硬哈希函数

6. API 安全6. API 安全

6.1 JWT 安全使用6.1 JWT 安全使用

```
const jwt = require('jsonwebtoken');

// JWT
const token = jwt.sign(
  {
    sub: user.id,
    role: user.role,
    jti: crypto.randomUUID(), //
  },
  process.env.JWT_PRIVATE_KEY,
  {
    algorithm: 'RS256', //
    expiresIn: '15m', //
    issuer: 'api.example.com',
    audience: 'example-app',
  }
);

// JWT
const decoded = jwt.verify(token, process.env.JWT_PUBLIC_KEY, {
  algorithms: ['RS256'],
  issuer: 'api.example.com',
  audience: 'example-app',
  maxAge: '15m',
});
```

6.2 API 速率限制6.2 API 速率限制

接口类型接口类型	限制策略限制策略	超限处理超限处理
注册/登录注册/登录	5 次/分钟/IP5 次/分钟/IP	返回 429 + 冷却时间返回 429 + 冷却时间
搜索搜索	30 次/分钟/用户30 次/分钟/用户	返回 429返回 429
数据修改数据修改	60 次/分钟/用户60 次/分钟/用户	返回 429返回 429
文件上传文件上传	10 次/分钟/用户10 次/分钟/用户	返回 429返回 429

7. 安全监控7. 安全监控

7.1 关键安全事件7.1 关键安全事件

事件类型事件类型	日志级别日志级别	告警方式告警方式
多次登录失败（同账户/IP）多次登录失败	WARNWARN	邮件 + 即时通讯邮件 + 即时通讯
权限越权访问权限越权访问	ERRORERROR	即时通讯 + 电话即时通讯 + 电话
SQL 注入尝试（WAF 拦截）SQL 注入尝试	WARNWARN	即时通讯即时通讯
异常流量（DDoS）异常流量（DDoS）	CRITICALCRITICAL	即时通讯 + 电话即时通讯 + 电话
敏感操作（删除/导出）敏感操作（删除/导出）	INFOINFO	审计日志审计日志

7.2 依赖漏洞扫描7.2 依赖漏洞扫描

```
# npm
npm audit --production

# Python
pip-audit

# OWASP Dependency-Check (Java)
mvn org.owasp:dependency-check-maven:check

#
trivy image my-app:latest
```

8. 安全检查清单8. 安全检查清单

上线前安全审查上线前安全审查

- ☐ 所有用户输入经过服务端验证
- ☐ 输出内容进行了上下文相关的编码
- ☐ HTTPS 已启用，HSTS 头已配置
- ☐ CSP 头已配置且无 unsafe-inline
- ☐ 敏感 Cookie 设置了 HttpOnly、Secure、SameSite
- ☐ 密码使用 bcrypt/argon2 哈希存储
- ☐ API 接口有速率限制
- ☐ CORS 配置正确，未使用 *
- ☐ 文件上传有类型和大小限制
- ☐ 依赖库无已知高危漏洞
- ☐ 错误信息不泄露内部实现细节
- ☐ 关键操作有审计日志

本指南由安全团队每季度审查更新，最新版本请查阅内部 Wiki。